
Desarrollo del *inflight software* para la OBC secundaria del Satélite Quetzal-2

Flavio André Galán Donis



UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



**Desarrollo del *inflight software* para la OBC secundaria
del Satélite Quetzal-2**

Trabajo de graduación presentado por Flavio André Galán Donis para
optar al grado académico de Licenciado en Ingeniería en Ciencias de la
Computación y Tecnologías de la Información

Guatemala, octubre

2026

Vo.Bo.:

(f) _____
Ing. Kuk Ho Chung

Tribunal Examinador:

(f) _____
Ing. Kuk Ho Chung

(f) _____
MSc. Carlos Esquit

(f) _____
Ing. Luis Pedro Montenegro

Fecha de aprobación: Guatemala, _____ de _____ de 2026.

Prefacio

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras vitae eleifend ipsum, ut mattis nunc. Pellentesque ac hendrerit lacus. Cras sollicitudin eget sem nec luctus. Vivamus aliquet lorem id elit venenatis pellentesque. Nam id orci iaculis, rutrum ipsum vel, porttitor magna. Etiam molestie vel elit sed suscipit. Proin dui risus, scelerisque porttitor cursus ac, tempor eget turpis. Aliquam ultricies congue ligula ac ornare. Duis id purus eu ex pharetra feugiat. Vivamus ac orci arcu. Nulla id diam quis erat rhoncus hendrerit. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Sed vulputate, metus vel efficitur fringilla, orci ex ultricies augue, sit amet rhoncus ex purus ut massa. Nam pharetra ipsum consequat est blandit, sed commodo nunc scelerisque. Maecenas ut suscipit libero. Sed vel euismod tellus.

Proin elit tellus, finibus et metus et, vestibulum ullamcorper est. Nulla viverra nisl id libero sodales, a porttitor est congue. Maecenas semper, felis ut rhoncus cursus, leo magna convallis ligula, at vehicula neque quam at ipsum. Integer commodo mattis eros sit amet tristique. Cras eu maximus arcu. Morbi condimentum dignissim enim non hendrerit. Sed molestie erat sit amet porttitor sagittis. Maecenas porttitor tincidunt erat, ac lacinia lacus sodales faucibus. Integer nec laoreet massa. Proin a arcu lorem. Donec at tincidunt arcu, et sodales neque. Morbi rhoncus, ligula porta lobortis faucibus, magna diam aliquet felis, nec ultrices metus turpis et libero. Integer efficitur erat dolor, quis iaculis metus dignissim eu.

Índice

Prefacio	3
Resumen	7
Abstract	8
1 Introducción	1
2 Antecedentes	3
3 Justificación	4
4 Objetivos	5
4.1 Objetivo general	5
4.2 Objetivos específicos	5
5 Alcance	6
6 Marco Teórico	7
7 Metodología	11
7.1 Ambiente de Desarrollo	11
7.2 Configuración del Hardware	11
7.2.1 I2C	12
7.2.2 UART	12
7.3 OBC Secundaria Quetzal-2	14
8 Conclusiones	16
9 Recomendaciones	17
Bibliografía	18
10 Anexos	21
10.1 Repositorio de Código	21
Glosario	22

Lista de figuras

Figura 1	Máquina de Estados Finitos (incompleta) de la OBC secundaria del Quetzal-2.	
	Elaboración propia.	14
Figura 2	Arquitectura de la OBC secundaria. Elaboración propia.	15

Lista de cuadros

Resumen

El Quetzal-2 es un nanosatélite CubeSat 2U desarrollado como proyecto académico por estudiantes y *staff* de la Universidad del Valle de Guatemala el cual pone a prueba una *On-Board Computer* (OBC) diseñada localmente que ejecuta una inteligencia artificial capaz de identificar nubes. Este nanosatélite cuenta con una computadora a bordo compuesta de por dos unidades de procesamiento, una OBC principal basada en hardware comercial de GomSpace y otra OBC secundaria desarrollada localmente utilizando Portentas H7 de Arduino.

Las OBC son sistemas vitales dentro de una misión espacial puesto que gestionan todos los subsistemas del satélite así como la comunicación entre estos. Este proyecto se centra en las bases de esta OBC local a nivel de *software*, sentando las bases para los distintos modos de operación que necesita la misión, la toma de fotografías y la interoperabilidad entre las dos unidades de procesamiento que componen la OBC completa.

El impacto de esta OBC desarrollada localmente representa un gran punto de inicio para próximas misiones espaciales, bajando aún más la barrera económica de entrada para desarrollar misiones espaciales con respecto a soluciones *of-the-shelf*, puesto que demuestra que no es necesario el uso de *hardware* altamente costoso con muchas más protecciones para llevar a cabo una misión exitosa.

Abstract

This is an abstract of the study developed under the

I

Introducción

El Quetzal-2 es un proyecto académico desarrollado por estudiantes y personal de la Universidad del Valle de Guatemala la cual pone a prueba una computadora a bordo (OBC por sus siglas en inglés) diseñada en UVG capaz de ejecutar un modelo de inteligencia artificial para identificar nubes en imágenes satelitales [1]. Este proyecto es importante no solo porque representa un avance tecnológico en Guatemala, sino también por su impacto dentro de la juventud del país, inspirando a futuros científicos, ingenieros e innovadores [2].

La misión espacial es un CubeSat 2U, 10x20x10cm y con un peso máximo de 4kg [3] y cuenta con 4 objetivos específicos que busca completar, llamadas cargas útiles (*Payloads* en inglés). Para este proyecto nos interesa específicamente el *inflight software* de la OBC secundaria de Quetzal-2 y su interacción con *Payload MILO*, el subsistema encargado del reconocimiento de nubosidad y la toma de fotografías en el espacio [1]. Es importante notar que estos 4 objetivos de misión son independientes de los 3 objetivos específicos de este trabajo profesional detallados en la sección de objetivos.

Las OBCs cumplen un papel clave dentro de los subsistemas del satélite puesto que son las encargadas de manejar todas las tareas, intercambio de información entre módulos y la colecta de información sobre los demás subsistemas (*housekeeping*) antes de la conexión con la estación en tierra [4].

Algunos de los subsistemas que debe manejar una OBC son:

- Un sistema de poder (EPS por sus siglas en inglés).
- Un sistema que determina la altitud (ADCS por sus siglas en inglés).
- Un sistema de comunicación utilizando radiofrecuencias.
- Una o más cargas útiles, por ejemplo una cámara o transmisor de señales junto con su controlador.

La inicialización, intercomunicación y gestión de todos estos subsistemas recae sobre la OBC [5]. Debido a las altas limitaciones energéticas en satélites de esta escala se debe tener especial cuidado con el presupuesto de energía que se le puede brindar a cada uno de los subsistemas. De esta necesidad nacen los modos de operación, los cuáles restringen las funcionalidades del satélite según la tarea para la que se quiere que se especialice en ese momento.

Este trabajo profesional busca desarrollar un sistema de *software* para la OBC diseñada localmente (*in-house*) capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de *handover* con la computadora principal, validado en un entorno controlado terrestre. Para el desarrollo de este sistema se utiliza una metodología propia

del laboratorio espacial UVG, basada en la metodología de ingeniería de sistemas de la NASA [6].

II

Antecedentes

Puede encontrarse un trabajo similar en [1] o bien [2].

III

Justificación

La misión espacial Quetzal-2 al ser una misión académica y no un proyecto de gobierno o una multinacional como Tesla, cuenta con recursos económicos relativamente limitados. Esto genera una necesidad real de reducir costos en donde sea posible, por lo tanto, un *inflight software* diseñado localmente, adaptado a las necesidades específicas de la misión pero reutilizable en misiones futuras y además es capaz de ejecutarse dentro de componentes más baratos resulta atractivo, dando origen a este proyecto.

En cuanto a las tecnologías a utilizar, las OBCs dependen en gran medida del microcontrolador seleccionado para su misión [7]. En el Quetzal-2 se utiliza una pareja de Portentash7 Lite para la OBC secundaria, éstas cuentan con dos procesadores ARM diseñados por el proveedor STM32 [8]. Por lo tanto, nuestras opciones se encuentran limitadas a lo que este proveedor pueda ofrecer.

Según la literatura actual, el uso de *Real Time Operating Systems* (RTOS por sus siglas en inglés) representa una práctica común para escribir sistemas complejos en el mundo del *embedded software*, incluyendo en misiones espaciales como el STUDSAT-2 [7]. Para este proyecto se utiliza FreeRTOS debido a su soporte para Portentah7 y su comunidad de desarrolladores, la cual es grande y activa [9]. Debido a que FreeRTOS expone un API oficial en C para su desarrollo [9] y a los bajos márgenes de presupuesto de energía con los que se cuenta en misiones de escalas similares a Quetzal-2 [4], el lenguaje que se utiliza para el desarrollo del *inflight software* es C.

IV

Objetivos

Objetivo general

Desarrollar localmente las bases de un *inflight software* para la *On Board Computer* (OBC) secundaria diseñada localmente para el Quetzal-2, capaz de operar la carga útil MILO, gestionar modos de operación y ejecutar un mecanismo de *handover* con la OBC principal, validado en un entorno controlado terrestre.

Objetivos específicos

1. Integrar el módulo de cámara al sistema de *software* de la OBC secundaria, logrando la captura de la imagen y transmisión de informaciones de la imagen desde la carga útil MILO hacia la OBC secundaria desarrollada localmente.
2. Desarrollar el sistema base de gestión de modos de operación del *software*, implementando tres modos: Arranque, Toma de fotografía y Nominal preliminar.
3. Desarrollar un Producto Mínimo Viable (MVP) del sistema de *handover* entre la OBC principal y la OBC secundaria diseñada localmente.

V

Alcance

Podemos usar Latex para escribir de forma ordenada una fórmula matemática.

VI

Marco Teórico

El Quetzal-2 es el segundo proyecto aeroespacial de la Universidad del Valle de Guatemala, el cual es desarrollado por estudiantes y personal académico de la institución [1].

Su predecesor, el Quetzal-1, fue el primer satélite guatemalteco, cuya misión buscaba probar un sensor multiespectral para adquirir información remota para conservar recursos naturales de forma independiente [2]. El trabajo realizado para Quetzal-1 representa las bases para el nuevo y mejorado Quetzal-2, el cual busca poner a prueba una computadora a bordo (OBC por sus siglas en inglés) diseñada localmente, capaz de ejecutar un modelo de inteligencia artificial para identificar nubes en imágenes satelitales. Así como validará un subsistema para desorbitación responsable y permitirá transmitir datos satelitales en tiempo real a centros educativos del país [1].

Debido a la complejidad de la misión, Quetzal-2 es un CubeSat 2U, el doble de tamaño que su predecesor [1]. CubeSat es un estándar que inició en 1999, desarrollado por el profesor Jordi Puig-Sauri y Bob Twiggs. La intención de este estándar es reducir costos y tiempos de desarrollo, al mismo tiempo que incrementa la accesibilidad al espacio. Todos los satélites CubeSat adoptan un tamaño y peso medido en unidades (“U”), la medida que define el estándar. Un CubeSat de 1U (como el Quetzal-1 [2]) es un centímetro de 10cm de lado con una masa de hasta 2kg [3].

Esta clase de misiones espaciales se componen de varios subsistemas comunicándose entre sí para llevar a cabo el objetivo de la misión, de esta realidad surge la necesidad de una OBC, la cual se encarga de manejar y dirigir de forma autónoma estas comunicaciones, así como la interacción con la estación de control terrena (GCS por sus siglas en inglés) [4].

Según Gildeh [10], un CubeSat típico se compone de varios subsistemas, como mínimo:

- Sistema de poder (EPS por sus siglas en inglés), cargado por paneles solares.
- Una carga útil (comúnmente llamada *payload*), el/los objetivo(s) de la misión, generalmente una cámara o similar.
- Sistema de control y determinación de altura (ADCS por sus siglas en inglés).
- Sistema de comunicación por radiofrecuencia.
- Una OBC que permite la comunicación entre todos estos subsistemas.

En el caso de Quetzal-2, la OBC realmente se divide en 2, una principal y una secundaria. La OBC principal, es la misma que se utilizó en Quetzal-1, la Gomspace Nanomind A3200 [11], esta computadora fue especialmente diseñada para misiones espaciales, como sensores integrados para control de altitud y protecciones especiales contra radiación [12].

La OBC secundaria, la cual es diseñada localmente, está compuesta por una pareja de Portentas H7 Lite, ambas cuentan con dos cores de procesamiento y 8MB de SDRAM [8]. La OBC secundaria, al igual que la primaria, necesita gestionar la comunicación entre los distintos subsistemas, por lo que se le considera el cerebro del satélite [4] y según [13] necesita cumplir con las siguientes tareas:

- Grabar y guardar la telemetría del satélite, incluyendo la data de las *payloads* y su transmisión hacia la GCS.
- Cifrado y descifrado de los paquetes de datos enviados y recibidos de la GCS.
- Monitoreo y gestión de subsistemas, incluso reiniciando los que sean necesarios.

Generalmente, la OBC no se encarga de el manejo de energía, en su lugar un subsistema por aparte (el EPS) se encarga de apagar y reiniciar sistemas dentro del satélite. Aunque sí hay casos en donde tal tarea se le atribuye a la OBC [14].

Existen varias interfaces seriales de comunicación que se pueden utilizar para la intercomunicación de microcontroladores, muchas aunque desarrolladas para una aplicación en específico se han vuelto universales, RS485 e I2C caen en esta categoría [15]. Dentro del Quetzal-2 se utilizan estos dos protocolos para la comunicación interna entre sus subsistemas, RS485 es especialmente popular debido a que permite conectar múltiples puntos de control utilizando un bus serial, además de ser un protocolo de comunicación probado en producción por años lo que lo hace una opción segura cuanto menos [16]. Mientras que I2C facilita la comunicación entre microcontroladores utilizando un modelo de maestro esclavo, en donde solo el maestro puede iniciar la comunicación [17].

Dentro de Quetzal-2, los protocolos se utilizan para los siguientes propósitos:

- RS485: Comunica la OBC primaria y secundaria con el resto de subsistemas del satélite, es decir: *Payload* MILO, *ADCS*, *ADM*, etc.
- I2C: Se reserva únicamente para la comunicación entre la OBC primaria y la OBC secundaria, siendo la OBC primaria la maestra y la OBC secundaria la esclava. Esta decisión es clave para el funcionamiento de la arquitectura de *handover*.

La arquitectura de *handover* es un sistema interno de la OBC primaria, diseñado localmente, cuyo objetivo principal es trasladar el control del satélite de sí misma a la OBC secundaria de una forma segura y redundante a fallos. Quetzal-2 no es la única misión que ha integrado más de una OBC en su sistema de mando, SHEFEX III, una misión alemana del 2017 también usó dos OBCs para redundancia, sin embargo su estrategia nunca fue ceder control de una OBC a la otra sino tener un backup en caso alguna de la dos fallara [18]. Otro ejemplo es la arquitectura DHS propuesta en 2023 para misiones académicas como profesionales [19]. Esta arquitectura en específico también se usó en el mismo año para otra misión espacial, la misión HERA, de la misma forma su principal objetivo era redundancia total, pero además buscaba autonomía operacional y alta capacidad de almacenamiento a bordo [20].

En Quetzal-2, esta arquitectura funciona de la siguiente manera:

La OBC diseñada localmente es desarrollada en un lenguaje a bajo nivel, particularmente C, debido a que es el lenguaje principal en el que el *driver* a bajo nivel de STM32 es

proveído, ofreciendo un mayor control sobre el uso de los recursos, casi tan granular como querramos [21]. Debido a la complejidad del sistema interno que debe manejar la OBC para gestionar la comunicación entre todos los subsistemas del satélite, se necesita un sistema operativo [4].

Existen muchos tipos de sistemas operativos (OS por sus siglas en inglés), según su infraestructura interna podemos tener sistemas operativos monolíticos, por capas, micro-kernels, módulos o híbridos siendo alguna combinación entre ellos [22]. Para propósitos del Quetzal-2, resulta de mayor importancia cómo el OS calendariza sus tareas que el cómo se compone internamente el sistema operativo como tal, ya que las misiones espaciales tienden a tener presupuestos muy ajustados tanto de memoria como procesamiento [4].

Los sistemas operativos en tiempo real (RTOS por sus siglas en inglés) fueron creados justamente para los casos en donde se tienen requerimientos de tiempo rígidos [22] y por esta razón se evaluaron distintas alternativas de implementación de un RTOS. RODOS, un sistema operativo de código abierto desarrollado por Sergio Montenegro, con un énfasis en facilidad de uso, rendimiento y probado en aplicaciones espaciales reales [23]; FreeRTOS, otro sistema operativo de código abierto, con una gran comunidad, excelente rendimiento y años de uso en producción [9]. Luego de unas pruebas de *porting* de RODOS a STM32H7, la arquitectura de la Portenta H7 Lite, se decidió a utilizar FreeRTOS por su alta compatibilidad con este *hardware*.

La arquitectura de software busca gestionar la comunicación entre los componentes de software que conforman un sistema [24]. En este caso, sistema se referiría a la OBC diseñada localmente. Al ser una misión espacial, el acceso que tendremos al satélite es muy limitado, no podremos realizar parches de actualización de código por ejemplo. Por lo que tener la mayor garantía que podamos de que el sistema funciona y es resiliente a fallos es la prioridad más alta.

La *National Aeronautics and Space Administration* (NASA) es famosa por sus 10 reglas para desarrollar código crítico seguro:

1. No usar recursión, saltos ni *goto statements*.
2. Todos los ciclos deben tener un límite superior fijo.
3. No alojar memoria dinámica luego de la inicialización.
4. No más de 60 líneas de código por función.
5. Cada función debe tener un promedio de dos *assertions*.
6. Declara todos los objetos de tipo data en el tamaño mínimo posible de *scope*.
7. Todos los parámetros de retorno deben ser revisados por la función que los llamó. Todas las funciones deben revisar la validez de los parámetros proveídos.
8. El uso del preprocesador de C debe estar limitado a inclusión de *header files* y definiciones sencillas.
9. Solamente es permitido un nivel de referencia con punteros.
10. Todo el código debe ser compilado desde el primer día de desarrollo con todas las alertas del compilador en la configuración más pedante. El código debe compilar sin advertencias.

Todas son importantes, pero esta última en especial aplica aunque la herramienta dé un falso positivo. Si esto sucede, el código debe ser reescrito para facilitar su análisis estático [25].

El espacio no es el único sector dentro del mundo del *software* que necesita una resiliencia extraordinaria. Las bases de datos, son piezas de *software* que deben funcionar incluso si el disco se corrompe, como es el caso de TigerBeetle [26]. U otra base de datos venerada por su resiliencia a lo largo de los años, SQLite, con más de 1 billón de instancias en uso actualmente [27].

Aunque hechas para casos de uso muy distintos, ambas llegaron a la misma conclusión, la forma de garantizar resiliencia a fallos, incluso en ambientes hostiles, es utilizar el poder de la misma computadora para revisar tu código. No hablan de IA, sino de *fuzz testing*, *unit testing* y otra gran variedad de *xxx testing*. No basta solo el análisis estático, hay que poder garantizar de forma automatizada que la solución funciona y es resiliente a fallos, no porque el *linter* no encuentre errores, sino porque luego de años de simulación que ocurren en horas o días en tiempo real lo respaldan [26], [27].

(me gustaría expandir mucho más en las reglas de la NASA y en estos dos casos de *software* resiliente pero me quedé sin tiempo para seguir escribiendo perdón Gabriel :«v)

VII

Metodología

Ambiente de Desarrollo

Para el desarrollo de las bases del software de vuelo de la OBC secundaria del Quetzal-2 se utilizó el sistema operativo libre Linux. La *distro* específica que se utilizó es irrelevante, pero si necesitas una recomendación, considero que Ubuntu Linux es una muy buena opción para principiantes.

1. Un editor de código. Yo utilicé Neovim (versión 0.12.4), pero perfectamente se puede usar Zed, VSCode o similares.
2. Instalar el manejador de paquetes Nix (versión 2.34.8).
3. Habilitar Nix Flakes.
4. Por último, si necesitas cambiar los valores de configuración por defecto de la portenta, vas a necesitar el STM32CubeIDE. Lo puedes obtener aquí (versión 2.2.0).

Copia y pega el archivo `flake.nix` del repositorio del proyecto (se encuentra en la Sección 10.1). Este archivo especifica todas las dependencias necesarias para compilar y quemar el proyecto en la memoria del microcontrolador.

Para instalar estas dependencias utilizando Nix, se debe abrir una terminal y ejecutar el siguiente comando dentro de la misma carpeta en la que está el `flake.nix`:

```
nix develop
```

Este comando creará una sesión con los paquetes necesarios para compilar el proyecto. Siempre ejecuta este comando antes de cualquier otro comando una vez antes de cualquier otro comando de compilación. Una lista incompleta de las dependencias que instalará es:

- gcc
- make
- bear
- clang
- Entre otras, el listado completo lo puedes encontrar dentro del archivo `flake.nix`.

Los pasos para compilar el proyecto desde 0 se pueden encontrar en detalle en el README del repositorio (Sección 10.1).

Configuración del Hardware

La OBC secundaria se encuentra compuesta por 2 *PortentasH7 Lite*. Ambas se configuraron utilizando el STM32H7CubeIDE para autogenerar el código de configuración. Específicamente, la OBC secundaria utiliza 2 tipos de comunicación serial: I2C y UART. Se configuraron de la siguiente manera:

I2C

Para la comunicación serial por I2C. La OBC secundaria es la esclava. Se utiliza el puerto I2C1 con la siguiente configuración:

```
I2C_HandleTypeDef hi2c1;

void MX_I2C1_Init(void) {

    hi2c1.Instance = I2C1;
    hi2c1.Init.Timing = 0x10C0ECFF;
    hi2c1.Init.OwnAddress1 = (0x09 << 1);
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK) {
        Error_Handler();
    }
    /** Configure Analogue filter
    */
    if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) != HAL_OK) {
        Error_Handler();
    }
    /** Configure Digital filter
    */
    if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK) {
        Error_Handler();
    }
}
```

UART

Para la comunicación utilizando RS485, se utiliza un puerto en específico de UART: El UART4. Se inicializa de la siguiente manera:

```
void MX_UART4_Init(void) {
    huart4.Instance = UART4;
    huart4.Init.BaudRate = 115200;
    huart4.Init.WordLength = UART_WORDLENGTH_8B;
    huart4.Init.StopBits = UART_STOPBITS_1;
    huart4.Init.Parity = UART_PARITY_NONE;
```

```

huart4.Init.Mode = UART_MODE_TX_RX;
huart4.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart4.Init.OverSampling = UART_OVERSAMPLING_16;
huart4.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
huart4.Init.ClockPrescaler = UART_PRESCALER_DIV1;
huart4.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
if (HAL_UART_Init(&huart4) != HAL_OK) {
    Error_Handler();
}
if (HAL_UARTEx_SetTxFifoThreshold(&huart4, UART_TXFIFO_THRESHOLD_1_8) !=
    HAL_OK) {
    Error_Handler();
}
if (HAL_UARTEx_SetRxFifoThreshold(&huart4, UART_RXFIFO_THRESHOLD_1_8) !=
    HAL_OK) {
    Error_Handler();
}
if (HAL_UARTEx_DisableFifoMode(&huart4) != HAL_OK) {
    Error_Handler();
}
}

```

La OBC secundaria también utiliza un puerto UART para enviar *logs* con información adicional que aumentan la observabilidad detrás de qué está pasando internamente dentro del sistema, los cuales se pueden leer leyendo el puerto UART6, configurado de la siguiente manera:

```

void MX_USART6_UART_Init(void) {
    huart6.Instance = USART6;
    huart6.Init.BaudRate = 115200;
    huart6.Init.WordLength = UART_WORDLENGTH_8B;
    huart6.Init.StopBits = UART_STOPBITS_1;
    huart6.Init.Parity = UART_PARITY_NONE;
    huart6.Init.Mode = UART_MODE_TX_RX;
    huart6.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart6.Init.OverSampling = UART_OVERSAMPLING_16;
    huart6.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
    huart6.Init.ClockPrescaler = UART_PRESCALER_DIV1;
    huart6.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
    if (HAL_UART_Init(&huart6) != HAL_OK) {
        Error_Handler();
    }
    if (HAL_UARTEx_SetTxFifoThreshold(&huart6, UART_TXFIFO_THRESHOLD_1_8) !=
        HAL_OK) {
        Error_Handler();
    }
    if (HAL_UARTEx_SetRxFifoThreshold(&huart6, UART_RXFIFO_THRESHOLD_1_8) !=
        HAL_OK) {
        Error_Handler();
    }
    if (HAL_UARTEx_DisableFifoMode(&huart6) != HAL_OK) {
        Error_Handler();
    }
}

```

```

    }
}

```

OBC Secundaria Quetzal-2

La OBC secundaria se diseñó a nivel de *software* para cumplir los siguientes objetivos, en orden de prioridad:

1. Fiabilidad
2. Seguridad
3. Rendimiento

La OBC secundaria se implementó como una máquina de estados finitos determinista, de esta forma podemos predecir no solo el rendimiento de memoria de la computadora secundaria, sino también su comportamiento. Algunos de los estados de la máquina finita se pueden apreciar en la Figura 1.

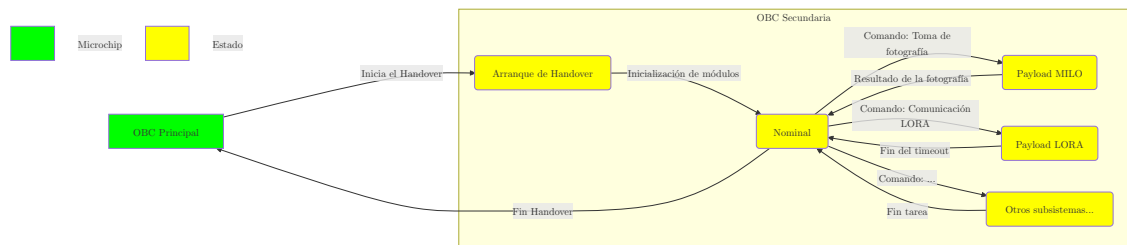


Figura 1: Máquina de Estados Finitos (incompleta) de la OBC secundaria del Quetzal-2.
Elaboración propia.

Para mejorar el determinismo de esta máquina de estados, todas las tareas que necesitan comunicación con un sistema externo al *core* de la OBC secundaria, por ejemplo interactuar con subsistemas, se realizaron por medio de interfaces. De esta forma se tiene una arquitectura como la descrita en la Figura 2.

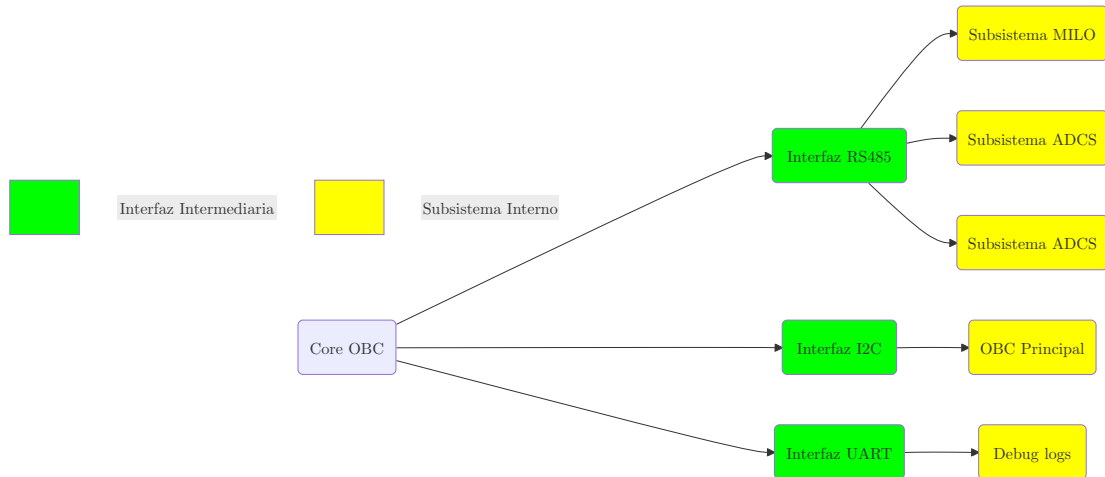


Figura 2: Arquitectura de la OBC secundaria. Elaboración propia.

Esta arquitectura de la Figura 2 nos permitió varias bondades:

- Nos permitió intercambiar todos los subsistemas (en amarillo) por implementaciones que funcionen o fallen de formas predecibles.
- Se logró simular entradas y salidas de los distintos subsistemas a voluntad sin necesidad de conectar físicamente todo el sistema o que siquiera se encuentre desarrollado.
- Se automatizaron las pruebas de comportamiento de la OBC ante una gran variedad de entradas de los distintos submódulos.

VIII

Conclusiones

IX

Recomendaciones

Bibliografía

- [1] U. del Valle de Guatemala, «Satélite CubeSat Quetzal-2». [En línea]. Disponible en: <https://www.uvg.edu.gt/quetzal-2/>
- [2] UVG, «CubeSat». [En línea]. Disponible en: <https://www.uvg.edu.gt/cubesat/>
- [3] A. Johnstone, *CubeSat Design Specification Rev. 14.1 The CubeSat Program, Cal Poly SLO CubeSat Design Specification Cal Poly -San Luis Obispo, CA Document Classification X Public Domain*. 2022. [En línea]. Disponible en: https://static1.squarespace.com/static/5418c831e4b0fa4ecac1bacd/t/62193b7fc9e72e0053f00910/1645820809779/CDS+REV14_1+2022-02-09.pdf
- [4] T. Lwabanji, R. Wilkinson, y E. Biermann Bellville, *Development of an OnBoard Computer (OBC) for a CubeSat*. 2013. [En línea]. Disponible en: https://etd.cput.ac.za/bitstream/20.500.11838/1172/1/Lumbwe_T_Final2013.pdf
- [5] S. A. Akhtar, *Design and Development of Obc of a Pico Spacecraft*. LAP Lambert Academic Publishing, 2012.
- [6] S. R. Hirshorn, *NASA Systems Engineering Handbook*. 2007. [En línea]. Disponible en: https://www.nasa.gov/wp-content/uploads/2018/09/nasa_systems_engineering_handbook_0.pdf?emrc=4096b9
- [7] B. Rajulu, S. Dasiga, y N. R. Iyer, «Open source RTOS implementation for on-board computer (OBC) in STUDSAT-2», *IEEE*, mar. 2014, doi: <https://doi.org/10.1109/aero.2014.6836377>.
- [8] Arduino, *Arduino® Portenta H7 Collective Datasheet*. 2026. [En línea]. Disponible en: <https://docs.arduino.cc/resources/datasheets/ABX00042-ABX00045-ABX00046-datasheet.pdf>
- [9] FreeRTOS, «FreeRTOS - Market Leading RTOS (Real Time Operating System) for Embedded Systems with Internet of Things Extensions». [En línea]. Disponible en: <https://www.freertos.org/>
- [10] D. Gildeh, *Final Report: Design and development of OBC for Pico-Sat PALMSAT*. 2003. [En línea]. Disponible en: https://davidgildeh.com/sites/davidgildeh.com/files/projects/Palmsat_Report.pdf
- [11] A. Aguilar-Nadalini *et al.*, «Design and On-Orbit Performance of the Electrical Power System for the Quetzal-1 CubeSat», *JoSS*, vol. 12, n.º 02, p. 1201, 2023, [En línea]. Disponible en: <https://jossoonline.com/storage/2023/05/Final-Aguilar-Nadalini-Design-and-On-Orbit-Performance-of-the-Electrical-Power-System-for-the-Quetzal-1-CubeSat.pdf>

- [12] Gomspace, *NanoMind A3200 - GomSpace*. 2026. [En línea]. Disponible en: <https://gomspace.com/product/nanomind-a3200/>
- [13] L. Stras *et al.*, «The design and operation of the Canadian advanced nanospace eXperiment (CanX-1)», en *Proc. AMSAT-NA 21st Space Symposium, Toronto, Canada*, 2003, pp. 150-160.
- [14] N. D. Hidayat, *Development of Flight Software for The Nanosatellite Onboard Computer Based on FM430 and Real-Time Operating System*. 2010.
- [15] P. D. Hung, V. Van Chin, N. T. Chinh, y T. D. Tung, «A Flexible Platform for Industrial Applications Based on RS485 Networks.», *J. Commun.*, vol. 15, n.º 3, pp. 245-255, 2020.
- [16] J. Sastry, A. Suresh, y S. Bhanu, «Building heterogeneous distributed embedded systems through rs485 communication protocol», *ARPJ Journal of engineering and applied sciences*, vol. 10, n.º 16, pp. 6793-6803, 2015.
- [17] E. J. Carletti, «Comunicación-bus i2c», *Robots Argentina*, 2007.
- [18] R. Schwarz y S. Theil, «A Fault-Tolerant On-Board Computing and Data Handling Architecture Incorporating a Concept for Failure Detection, Isolation, and Recovery for the SHEFEX III Navigation System», en *SpaceOps 2014 Conference*, 2014, p. 1874.
- [19] J.-F. Soucaille, «I. High Dependability Data Handling Systems for In Orbit Operations», en *2023 European Data Handling & Data Processing Conference (EDHPC)*, 2023, pp. 1-4.
- [20] D. M. Marcos y A. Valverde Carretero, «DHS architecture for HERA Deep Space Mission». [En línea]. Disponible en: <http://dx.doi.org/10.23919/EDHPC59100.2023.10396073>
- [21] STMicroelectronics, *STM32H7 - Arm Cortex-M7 and Cortex-M4 MCUs (480 MHz) - STMicroelectronics*. 2026. [En línea]. Disponible en: <https://www.st.com/en/microcontrollers-microprocessors/stm32h7-series.html>
- [22] Abraham Silberschatz; Greg Gagne; Peter B. Galvin, *Operating System Concepts*. Wiley Global Education, 2018.
- [23] O. Schuele, M. Rafat, y V. Kossykh, «The software environment of RODOS», pp. p. 1109-1118, 1996.
- [24] C. B. Pareja Valerio y L. J. Burgos Robles, «La arquitectura de software basada en microservicios: Una revisión sistemática de la literatura», 2019.
- [25] G. Holzmann, «The Power of 10: Rules for Developing Safety-Critical Code», en *Software Technologies*, 2006. [En línea]. Disponible en: <https://web.eecs.umich.edu/~imarkov/10rules.pdf>
- [26] J. D. Greef, «A Trillion Transactions». [En línea]. Disponible en: <https://tigerbeetle.com/blog/2026-03-19-a-trillion-transactions/>

- [27] S. S. Work, «Reliability Lessons From SQLite - Richard Hipp | SSW 2026». [En línea].
Disponible en: https://www.youtube.com/watch?v=V_qzqY1bb7I

X

Anexos

Repositorio de Código

El repositorio es público y se encuentra hosteado en github, da click [aquí](#).

Glosario

Latex

Es un lenguaje de marcado adecuado especialmente para la creación de documentos científicos

Fórmula

Una expresión matemática